DRAFT - NOT FINAL

# Phylaxtag Warranty

Design & build guide, version 3.5

*"The product owns its warranty."*

This e-bike has its own warranty fund. Sell the bike, and the fund and the service book move with it.

Name: Phylaxtag (Greek φύλαξ, "guardian"). Repo: phylaxtag · X: @phylaxtag · phylaxtag.com (both to be registered)
NEAR Hackathon 2026 (1-14 Nov 2026) · Track: NFTs That Do Something (nfts), optionally Real-World Money (realworld)
Prototype on NEAR testnet

Project owner: pranesta kresa · Spec owner: Pharita (Grok team)
Draft date: 4 November 2026 (WIB) · Author: Pharita · Replaces: Phylaxtag Warranty v3.4 (4 Nov 2026)

**How to use this document.** Sections 1-4 are for everyone, including judges. Sections 5-8 are the technical spec the Cursor team builds from. Sections 9-13 are the team plan, schedule and submission material. Anything not yet checked is marked **[UNVERIFIED]**; any number that must come from a real run is marked **[TO BE MEASURED]**. **Submission deadline: 14 Nov 2026 23:59 UTC = 15 Nov 2026 06:59 WIB.** Target submit: around 12 Nov.

**DRAFT - not final.** Only Pharita edits this spec. Comments and change requests go to Pharita. This document contains no keys, seed phrases or other secrets, and none may be added.

# Contents

Changes from v3.4 (v3.5)
Changes from v3.3 (v3.4)
Changes from v3.2 (v3.3)
Changes from v3.1 (v3.2)
Changes from v3 (v3.1)
Changes from v2
Changes from v1 (summary)
1. Pitch and one-line story
2. Problem
3. Solution and actors
4. End-to-end flow (example: e-bike EB-001)
5. Contract design
6. Threat model (incl. transfer threat model)
7. Test plan
8. Web pages
9. Team plan (three teams)
10. Day-by-day schedule
11. Demo video script (60-90 s)
12. Hackathon rules and submission checklist
13. Submission text (English)
14. Open items and unverified claims

# Changes from v3.4 (v3.5)

Resolves the 9 spec issues reported by the Cursor team in PR #1 (decided by Pharita). Test count unchanged: 39.

| # | Issue (PR #1) | Decision | Where |
|---|---|---|---|
| 1 | String limits not in mint checks | Mint checks `token_id` 1..32 bytes ("Bad token_id") and `serial_hash` = 64 lowercase hex chars ("Bad serial_hash"); `work_type` / `note_hash` stay on `pay_service` ("Bad work_type", "Bad note_hash", T30) | 5.1, 5.4, T29 |
| 2 | Display status undefined | Mapping from stored status, balance and current time (table 5.1b) | 5.1b, 8 |
| 3 | Next claim time undefined | `next_claim_ns` = null before the first claim (claim possible now), else `last_claim_at_ns + cooldown_ns`; `last_claim_at_ns` becomes `Option<u64>` | 5.1b, 5.3 |
| 4 | Duplicate workshops | Rejected with "Bad workshop list" (extra case inside T29) | 5.1, T29 |
| 5 | Callback / history signatures | Defined in table 5.1a | 5.1a |
| 6 | Contract-level NEP-177 metadata | Constants, not stored: name "Phylaxtag", symbol "PHYLAX", spec = the crate's NEP-177 version string, no icon, no base_uri | 5.6 |
| 7 | mint return value | Returns the minted NEP-171 `Token` | 5.1 |
| 8 | mint and the holder log | Mint writes the first HolderRecord (issuer, mint time, `holder_total` = 1) and an empty service history | 5.1, 5.3 |
| 9 | Transfer hook | No hook in near-contract-standards 5.30.0: own `NonFungibleTokenCore::nft_transfer` calls the inner transfer, then appends the holder record. Pinned: near-sdk 5.30.0, near-contract-standards 5.30.0, near-workspaces 0.24.0 | 5, 5.6, F5 |

## Changes from v3.3 (v3.4)

| # | Change | Where |
|---|---|---|
| 1 | Rename only, no design change: project is now **Phylaxtag** (Greek φύλαξ, "guardian"). Repo `phylaxtag`, X @phylaxtag and phylaxtag.com to be registered. O5 resolved. | All, 1, 12, 13, 14 |

## Changes from v3.2 (v3.3)

| # | Change (reviewer Jiwoo, user approved) | Where |
|---|---|---|
| 1 | T17: `total_locked` changes only via mint, pay/expire and their failed-payout callbacks (T10, T19, T21, T22); 5.1 rows for mint, pay_service and expire now state their `total_locked` effect; Day 0 says spec v3.3 and repo "new recommended" | 7, 5.1, 10 |
| 2 | S11 Test column: T28 covers same issuer only; cross-issuer duplicate is an accepted residual risk | 6.2, 6.3 |

## Changes from v3.1 (v3.2)

| # | Change (reviewer Jiwoo, user approved) | Where |
|---|---|---|
| 1 | Failed-payout / failed-refund callbacks restore the token balance **and** `total_locked` by the same amount (keeps the T26 invariant); added to T10, T19, T21, T22 | 5.1, 7 |
| 2 | Another allowlisted issuer could mint the same serial_hash: two warranty NFTs for one item. Accepted risk; asset page shows the issuer prominently | 6.2 S11, 6.3, 8 |
| 3 | Storage reserve top-up = plain NEAR transfer to the contract account (not the removed per-token `top_up`) | 5.4 |
| 4 | Nits: Day 0 "spec v3.2 approved"; Cursor row "one repo (new public repo recommended, not mandatory per Q-003)"; version labels | 9, 10 |

## Changes from v3 (v3.1)

Small edit pass applying five findings from reviewer **Jiwoo** (user approved).

| # | Change | Where |
|---|---|---|
| 1 | `mint` wording: workshops must be 1..N accounts, none of them the issuer (empty list or issuer as workshop fails, as in T29) | 5.1 |
| 2 | New check: a workshop must not be the contract's own account (protects the T26 invariant); new test T39; total now 39 tests | 5.1, 7 |
| 3 | Accepted risk added: no `remove_issuer` / `set_admin` in the MVP | 6.3, 14 |
| 4 | "New repo" marked OPTIONAL, quoting live Q-003; new repo stays our recommended option | 12 |
| 5 | README outline #3 now names exact test counts | 12.2 |

## Changes from v2

v3 applies the red-team findings of reviewer **Jiwoo** (Hermes team), approved by the user and written by Pharita (spec owner).

| # | Change | Where |
|---|---|---|
| A | Full submission API contract quoted verbatim from the live HACKATHON.md (fetched 2026-11-04): endpoint, every field rule, status codes, consent notice, spam-trap field, retry rules. Track ids re-verified (`nfts` is valid). Submission-text length recounted programmatically. | 12, 13 |
| B1 | `mint` rejects `cooldown_ns == 0`. In-flight guard considered and **not** adopted (justified in 5.2); new tests T34, T35. | 5.1, 5.2, 7 |
| B2 | `expire` always marks the token Expired after expiry, even with an empty fund (no transfer when nothing to refund). A second call is a defined no-op returning 0. `expire_in_flight` flag removed. New UI state "Empty (fund used up)". T09, T21, T22 updated; new T36. | 5.1, 5.3, 8, 7 |

| # | Change | Where |
|---|---|---|
| B3 | Holder log can never block a transfer: drop-oldest ring buffer of the latest 50 entries + a total-transfers counter. New T37. | 5.3, 5.4, 6, 7 |
| B4 | New residual risk: a seller can make a last claim right before selling, so the buyer waits a full cooldown. UI + README disclosure, no contract change. New web test T38. | 6.3, 8 |
| C | Similar-project check recorded (reviewer-reported, not independently verified); code-window rule and historic judging criteria recorded (reviewer-reported) and used as pitch ordering. | 1, 12, 14 |
| D | Open items split into Decided vs Still open. | 14 |

## Changes from v1 (summary)

| # | Change | Why |
|---|---|---|
| 1 | `top_up` **removed entirely.** The fund is deposited once, at mint, by the issuer. No later deposits by anyone. | User decision 4 Nov 2026. Keeps the money flow simple: issuer in, workshops or issuer out. Removes the "who owns a top-up?" confusion. |
| 2 | **Claim limits are automatic and enforced only by the contract:** per-claim cap, cooldown between claims, cumulative cap = remaining fund, issuer-set workshop allowlist, expiry. **No issuer co-sign or approval step.** | User decision 4 Nov 2026. One-call claims stay; residual collusion risk is documented honestly (section 6.3). |
| 3 | **New transfer threat model** (every way a seller could keep money or access), each with mitigation and test; new tests for a sale in the middle of a payout, including the payout-callback failure path. | Required by the official NFT-track prompt (`prompts/10-nfts.md`). |
| 4 | **New web pages:** holder-only (gated) page, per-NFT history (funded / paid out / who held it), "Sell this item" demo button, demo mode with short cooldown and expiry. | Required by the NFT-track prompt; makes the transfer visible in the demo. |
| 5 | **Fully English**, renamed working document "Phylaxtag Warranty v2". | Judges, AI pre-screeners and README are in English. |
| 6 | **Three-team structure** (Cursor = only code writer; Grok = spec, schedule, rules, submission; Hermes = red team, README, newcomer test) and hard rules on who may change what. | Avoids conflicting edits and keeps secrets with the user only. |
| 7 | Smaller fixes: exact storage accounting (explicit `fund_amount`, excess refunded), salted serial hash, 50-record cap, `nft_transfer_call` disabled, claims blocked while the token is listed (has approvals), owner may not pay themselves, `expire` can be retried if a refund comes back. | Found while writing the transfer threat model. |

## 1. Pitch and one-line story

**One line for a newcomer:** This e-bike has its own warranty fund. Sell the bike, and the fund and the service book move with it.

### 30-second pitch

Buying used goods is a trust problem. Warranties usually die when an item is resold, and service records are lost or live on paper.

Phylax (φύλαξ) is Greek for 'guardian': the warranty is a guardian that lives on the item and transfers with it.

Phylaxtag turns one physical item (MVP: one e-bike) into an NFT on NEAR. The NFT **carries a warranty fund and a service history**. The issuer (seller or manufacturer) deposits the fund once and chooses the approved workshops. The owner pays an approved workshop straight from the fund with one call, within limits the contract enforces. The owner can never withdraw the fund to their own wallet.

When the bike is sold, the NFT moves to the buyer, and the remaining fund and the full service history move with it. The seller loses access at that moment. After the warranty expires, the remaining fund goes back to the issuer.

**Pitch ordering (suggestion).** Reviewer Jiwoo reports that historic NEAR hackathon judging criteria were: technical implementation, design, impact, originality, sustainability **(reviewer-reported; not in the live HACKATHON.md)**. Order the pitch, README intro and video the same way: working contract + tests first, then the UX, then who benefits, then what is new, then how it could continue.

### Key facts

| Item | Value |
|---|---|
| Name | Phylaxtag. Phylax (φύλαξ) is Greek for 'guardian': the warranty is a guardian that lives on the item and transfers with it. |
| Repo / X / domain | `phylaxtag` / @phylaxtag / phylaxtag.com (X handle and domain to be registered) |
| Tagline | "The product owns its warranty." |
| Standards | NFT NEP-171 core, NEP-177 metadata, NEP-181 enumeration, NEP-178 approvals (restricted, see 5.6), NEP-297 events |
| Network | NEAR testnet (prototype) |
| MVP asset | One e-bike, token `EB-001` |
| Track | NFTs That Do Something (`nfts`); optionally Real-World Money (`realworld`) |
| Core calls | `mint`, `pay_service`, NEP-171 `nft_transfer`, `expire`; views `get_asset`, `get_service_history`, `get_holder_history` |
| Off-chain parts | No keeper or bot. Static web app + proof files off-chain (only hashes on-chain) |
| Claim to use | "To our knowledge, a working NFT-warranty implementation on NEAR." Never "the first" (section 14). |

## 2. Problem

- **Warranties do not follow the item.** They are tied to the first buyer's name or receipt. On resale the warranty is effectively gone.
- **Service history cannot be checked.** A used-item buyer cannot see what was serviced, when, or where.
- **Warranty value is invisible.** Nobody can see how much repair budget is left.
- **Claims are slow.** Forms, approvals and waiting are too heavy for small repairs.
- **Fraud risk.** If a warranty fund could be cashed out freely, an owner could stage fake repairs with a friend and drain it.

MVP users: an e-bike seller (issuer), the bike owner, approved workshops, and used-bike buyers.

## 3. Solution and actors

The warranty fund is stored **per token** in the contract, not per owner. The money is attached to the item, whoever holds it. Control is simply "whoever `nft_token(id).owner_id` says right now".

### Actors

| Actor | Who | May | May not |
|---|---|---|---|
| Admin | Contract deployer (the user's testnet account) | Set the issuer allowlist at `new`; `add_issuer` | Touch any token's fund |
| Issuer | Allowlisted seller / manufacturer | Mint with a one-time deposit; set workshops, expiry, per-claim cap, cooldown; receive the remainder after expiry | Take the fund back before expiry; change limits or workshops after mint; add money later |

| Actor | Who | May | May not |
|---|---|---|---|
| Owner | Current NFT holder | Call `pay_service` to an approved workshop within limits; sell (transfer) the NFT; open the holder-only page | Withdraw the fund; pay a non-approved account; pay themselves; claim while the token is listed |
| Workshop | Account on the token's allowlist | Receive payments | Trigger a payment |
| Buyer | Next owner | Inspect fund and history before buying; becomes owner on transfer | - |
| Anyone | Any account | Call `expire` after expiry; read all views | Change asset data |

### Design principles

- **One call per service.** No submit/approve stages. The owner calls `pay_service` and the contract checks the limits.
- **Money never reaches the owner.** It can only go to an approved workshop, or back to the issuer after expiry.
- **No pending state.** Nothing is "in progress" between calls except an in-flight payout, whose result always lands back on the token (section 6.2).
- **Limits are code, not people.** Cap, cooldown, remaining fund, allowlist and expiry. No co-signer.
- **One deposit.** The issuer funds the warranty once at mint. No `top_up`.

### Why not "Phylaxtag Lite" (owner-funded savings)

An owner-funded service wallet was considered and rejected: that is personal savings, not a warranty. The value of a warranty comes from the seller's promise, so the **issuer funds it**. With `top_up` removed in v2, nobody else can add money, so there is no confusion about who owns extra funds.

## 4. End-to-end flow (example: e-bike EB-001)

Example values: fund **10 NEAR**, one service **2 NEAR**, per-claim cap **3 NEAR**, cooldown **30 days** (demo token: short, e.g. 60 s), expiry **2027-11-03** (demo token: a few minutes). All example values are illustrative, not measured.

| # | Step | Who | Call | Result |
|---|---|---|---|---|
| 1 | Admin allowlists the issuer | Admin | `new(admin, [issuer])` or `add_issuer` | Issuer may mint |
| 2 | Issuer mints EB-001 and deposits the fund | Issuer | `mint("EB-001", serial_hash, 10 NEAR, 2027-11-03, 3 NEAR, 30d, [ws-a, ws-b], meta)` + attached 10 NEAR + storage cost | Fund exactly 10 NEAR; storage paid from the deposit; excess refunded to issuer |
| 3 | Issuer transfers the NFT to the first buyer, Alice | Issuer | `nft_transfer(alice, "EB-001")` | Owner = Alice; holder log: Alice, fund 10 |
| 4 | Alice services the bike at ws-a and pays from the fund | Alice | `pay_service("EB-001", ws-a, 2 NEAR, note_hash, "brakes")` | ws-a receives 2 NEAR; fund 8; 1 record; next claim after cooldown |
| 5 | Alice tries 4 NEAR, another workshop, or a second claim inside the cooldown | Alice | `pay_service(...)` | Rejected: "Over cap" / "Workshop not approved" / "Cooldown active" |
| 6 | Alice sells to Budi ("Sell this item" button) | Alice | `nft_transfer(budi, "EB-001")` | Owner = Budi; fund 8 and 1 record move; holder log: Budi, fund 8; Alice's holder page now says "not the holder" |
| 7 | Alice tries to pay again | Alice | `pay_service(...)` | Rejected: "Not owner". Budi can claim once the cooldown (which is per token) has passed |
| 8 | After 2027-11-03, anyone calls expire | Anyone | `expire("EB-001")` | Remaining fund (e.g. 8 NEAR) goes to issuer; status Expired; history stays readable |

# 5. Contract design

Rust with `near-sdk` and `near-contract-standards`. **Pinned** (checked 4 Nov 2026, recorded in `docs/DECISIONS.md`, PR #1): near-sdk 5.30.0, near-contract-standards 5.30.0, near-workspaces 0.24.0, Rust 1.99.1; `Cargo.lock` committed. Fallback if the pin breaks the build or sandbox tests: near-sdk / near-contract-standards 5.29.1 and near-workspaces 0.23.0 (compatibility of that pair not proven); the switch and its reason go into `docs/DECISIONS.md` and the README. All amounts are yoctoNEAR (1 NEAR = 10^24 yocto); all times are block timestamps in nanoseconds.

## 5.1 Function table

| Function | Caller | Effect | Checks (in order) | Errors (examples) |
|---|---|---|---|---|
| `new(admin, issuers)` init | Deployer | Set admin and issuer allowlist | Once only | "Already initialized" |
| `add_issuer(account)` | Admin | Add an issuer | caller = admin | "Only admin" |
| `mint(token_id, serial_hash, fund_amount, expiry, claim_cap, cooldown_ns, workshops, metadata)` payable | Allowlisted issuer | Create NFT owned by issuer; store asset; balance = `fund_amount` (`total_locked` + same); write first HolderRecord (issuer, mint time, `holder_total` = 1, `holder_next` = 1) and an empty service history; storage paid from deposit; excess refunded to issuer. **Returns the minted NEP-171 `Token`** | issuer allowlisted; `token_id` 1..32 bytes; `serial_hash` exactly 64 lowercase hex chars; token_id new (never reused); serial_hash new for this issuer; expiry > now; 0 < claim_cap ≤ fund; **cooldown_ns > 0**; workshops: 1..N accounts, no duplicates, none of them the issuer or the contract account (`env::current_account_id()`); deposit ≥ fund + storage cost | "Not an approved issuer", "Bad token_id", "Bad serial_hash", "Token exists", "Serial already used", "Bad expiry", "Bad cap", "Cooldown must be > 0", "Bad workshop list", "Deposit too small" |
| `pay_service(token_id, workshop, amount, note_hash, work_type)` payable 1 yocto | Current owner only | Deduct balance (and `total_locked`), append record, set `last_claim_at_ns = Some(now)`, send NEAR to workshop, callback `on_payout` | exactly 1 yocto; caller = owner; not expired; no active approvals; workshop on list; workshop ≠ owner; 0 < amount ≤ cap; amount ≤ balance; `last_claim_at_ns` is None or now ≥ last_claim_at_ns + cooldown; records < 50; `work_type` ≤ 32 bytes; `note_hash` = 64 lowercase hex | "Not owner", "Expired", "Token is listed", "Workshop not approved", "Cannot pay yourself", "Over cap", "Insufficient balance", "Cooldown active", "History full", "Bad work_type", "Bad note_hash" |
| `nft_transfer` | Owner or approved account | Standard NEP-171 move (`total_locked` unchanged); fund and records stay on the token; holder log written to the ring buffer (drop-oldest), `holder_total` + 1 | Standard (1 yocto); approvals cleared; **never fails because of the log** | Standard |
| `nft_transfer_call` | - | **Disabled in MVP** | Always panics | "nft_transfer_call disabled in MVP" |
| `expire(token_id)` → U128 refunded | Anyone | Set status Expired (always). If balance > 0: balance → 0 (`total_locked` − same), send it to issuer, callback `on_expire_payout`. If balance = 0: no transfer | now ≥ expiry. Never fails for "empty" or "already expired" | "Not yet expired", "Token not found" |
| `on_payout` / `on_expire_payout` | Contract itself (`#[private]`) | On failure: add the amount back **to the token balance and to `total_locked`** (both by the same amount, else the T26 invariant breaks), mark record Failed (payout), emit event; status stays as is | `#[private]`; reads promise result | - |
| `get_asset(token_id)` view | Anyone | Asset + owner + balance + stored status + **display status** and `next_claim_ns` (both per 5.1b) | - | null if missing |
| `get_service_history(token_id, from_index, limit)` view | Anyone | Paginated service records (signature in 5.1a) | - | null if token missing |

| Function | Caller | Effect | Checks (in order) | Errors (examples) |
|---|---|---|---|---|
| `get_holder_history(token_id)` view | Anyone | Latest ≤ 50 holders in time order (oldest first), with since and balance at transfer, plus `holder_total` (signature in 5.1a) | - | null if token missing |
| `get_storage_info()` view | Anyone | `storage_usage`, `total_locked`, contract balance (for the storage test) | - | - |

### 5.1a Signatures not shown above

| Function | Parameters | Returns |
|---|---|---|
| `on_payout #[private]` | `token_id: TokenId, workshop: AccountId, amount: NearToken, record_index: u64` | `bool`: true if the transfer succeeded (record → Paid); false: amount restored to balance and `total_locked`, record → Failed, `payout_failed` |
| `on_expire_payout #[private]` | `token_id: TokenId, amount: NearToken` | `bool`: true if the refund reached the issuer; false: amount restored to balance and `total_locked`, `expire_refund_failed` |
| `get_service_history` | `token_id: TokenId, from_index: Option<u64>` (default 0), `limit: Option<u64>` (default and max 50) | `Option<Vec<ServiceRecord>>` in record order; null if token missing; empty if `from_index` ≥ length |
| `get_holder_history` | `token_id: TokenId` | `Option<HolderHistory>` with `records: Vec<HolderRecord>` (≤ 50, oldest first) and `holder_total: u32`; null if token missing |

### 5.1b Display status and next claim (computed in get_asset)

| Stored status | Condition (now = block timestamp) | Display status |
|---|---|---|
| Active | now < expiry and balance > 0 | Active |
| Active | now < expiry and balance = 0 | Empty (fund used up) |
| Active | now ≥ expiry (any balance; `expire` not called yet) | Expired, close pending |
| Expired | balance > 0 (failed refund or late failed payout restored) | Expired, close pending |
| Expired | balance = 0 | Expired |

`next_claim_ns: Option<u64>` = null if `last_claim_at_ns` is None (no claim yet; a claim is possible now), else `last_claim_at_ns + cooldown_ns`. It is informational: a claim also needs display status Active. `last_claim_at_ns` is an `Option<u64>` so a claim at block timestamp 0 (unit tests) cannot be confused with "no claim yet".

There is **no** `top_up`, no owner withdrawal, and no function that changes workshops (so the workshop checks live only in `mint`; any future workshop-update function must repeat them), cap, cooldown or expiry after mint. The only payable entry points that keep money are `mint` (once per token) and the 1-yocto confirmations.

## 5.2 Claim-limit rules (automatic, contract-only)

| Rule | Set by | Enforced in | Demo-mode value |
|---|---|---|---|
| Per-claim cap: `amount ≤ claim_cap` | Issuer at mint | `pay_service` | e.g. 3 NEAR |
| Cooldown: `now ≥ last_claim_at + cooldown_ns`, `cooldown_ns > 0` required at mint (per token, **not reset on transfer**) | Issuer at mint | `pay_service` | short, e.g. 60 s |
| Cumulative cap: total paid ≤ remaining fund (balance can never go negative; no top-ups) | Implicit | `pay_service` | 10 NEAR fund |
| Workshop allowlist (fixed at mint; owner never on it as payee) | Issuer at mint | `pay_service` | 2 test workshop accounts |
| Expiry: no claims at/after expiry; remainder returns to issuer | Issuer at mint | `pay_service`, `expire` | a few minutes |
| No claims while the token has active approvals (listed for sale) | Contract rule | `pay_service` | - |

"Demo mode" is **not a contract flag**: it is just a demo token minted with short cooldown and expiry values. The contract has one code path for real and demo tokens. Upper bound on what an owner plus one colluding approved workshop can extract: `min(balance, claim_cap × (1 + floor((expiry − now) / cooldown)))`. The issuer chooses all three numbers; `cooldown_ns > 0` keeps the formula defined.

### Concurrency: why cooldown_ns > 0 and no in-flight flag

Calls to one contract run one at a time, and `pay_service` deducts the balance, writes the record and sets `last_claim_at` **before** it creates the payout promise. With `cooldown_ns > 0`, a second claim in the same block (same timestamp) or before the cooldown ends is rejected, and the balance can never be spent twice. The only asynchronous part is the payout itself; if it fails, the callback returns the amount to the token.

| Option | Pro | Con | Decision |
|---|---|---|---|
| Layer 1: require `cooldown_ns > 0` + state-before-promise | Simple; no extra state; nothing can get stuck | A very small cooldown still allows quick repeat claims (bounded by cap and balance) | **Adopted** |
| Layer 2: per-token `payout_pending` flag set in `pay_service`, cleared in `on_payout` | Blocks a new claim until the previous payout has settled | If the callback does not run as planned (e.g. a gas mistake), the token is frozen for claims; fixing that needs an admin unlock, which is a new trust and attack surface. Adds no money safety because the balance is already deducted | **Not adopted** in the MVP; the ServiceRecord `Pending` status is shown in the UI instead |

Callbacks must reserve enough gas so they always run; T10 and T19 check this. A minimum cooldown value for real tokens (beyond `> 0`) is an open item (section 14).

## 5.3 Storage layout

```
struct Asset {
    issuer: AccountId,
    serial_hash: String,          // sha256(salt || serial), salt kept off-chain by issuer
    balance: NearToken,           // the warranty fund, stored per token
    expiry_ns: u64,
    claim_cap: NearToken,
    cooldown_ns: u64,
    last_claim_at_ns: Option<u64>, // None until first claim; survives transfers
    workshops: Vec<AccountId>,    // fixed at mint
    status: Active | Expired,     // display adds Empty / close pending
    holder_total: u32,            // all transfers ever, incl. mint
    holder_next: u32,             // ring-buffer write position
}
struct ServiceRecord { workshop, amount, work_type, note_hash, owner_at_claim,
                       timestamp_ns, block_height, status: Pending | Paid | Failed }
struct HolderRecord  { owner, since_ns, balance_at_transfer }
Contract {
    tokens: NonFungibleToken,                          // NEP-171/177/181/178
    assets: LookupMap<TokenId, Asset>,
    history: LookupMap<TokenId, Vector<ServiceRecord>>,   // max 50
    holders: LookupMap<TokenId, Vector<HolderRecord>>,    // ring, latest 50
    used_serials: LookupSet<(AccountId, String)>,
    issuers: LookupSet<AccountId>,  admin: AccountId,  total_locked: NearToken,
}
```

**Holder log when full (decision):** the log is a drop-oldest ring buffer. Entries 1-50 are appended; from the 51st transfer on, the oldest slot is overwritten and `holder_total` keeps counting. The issuer and mint time stay in `Asset`, and every transfer remains in the permanent `nft_transfer` events, so full history can still be rebuilt by an indexer. A transfer therefore **never** fails because of the log, so the bike always stays sellable.

Key point: `balance` lives inside `Asset`, looked up by `token_id`. There is no per-owner balance, so nothing has to "move" on transfer. Test T01 proves it.

## 5.4 Storage accounting (exact)

- `mint` takes an explicit `fund_amount`. The contract measures `env::storage_usage()` before and after writing the token, computes the exact storage cost, requires `deposit ≥ fund_amount + cost`, and refunds the excess to the issuer. The demo fund is therefore exactly 10 NEAR.
- Service records and holder records are paid from a contract storage reserve funded by the user at deploy. To add more reserve later, send a plain NEAR transfer to the contract account: no function is needed, and the invariant stays safe because the balance only increases. This is **not** the removed `top_up` (which was per-token warranty funding); reserve money is never part of any token's fund. Both are capped at 50, so worst-case storage per token is bounded. After the holder ring is full, each transfer overwrites one slot: storage changes only by the size difference between the old and new account id, and never grows past 50 records of maximum account-id length.
- Invariant checked by test T26: `contract balance ≥ total_locked + storage_usage × storage_byte_cost`.
- Bytes per mint, per service record and per holder record, and the resulting NEAR cost: **[TO BE MEASURED]** by T26 (and holder-ring figures by T37) on Day 2, then copied into the README. No figures are given here on purpose.
- String limits (enforced, see 5.1): `token_id` 1..32 bytes and `serial_hash` = 64 lowercase hex chars in `mint`; `work_type` ≤ 32 bytes and `note_hash` = 64 lowercase hex chars in `pay_service`.

## 5.5 Events (NEP-297)

| Event | When | Data |
|---|---|---|
| `nft_mint`, `nft_transfer` (standard) | Mint / transfer | standard fields |
| `warranty_funded` | Mint only (no top-ups exist) | token_id, amount, issuer |
| `service_paid` | `pay_service` | token_id, owner, workshop, amount, work_type, note_hash |
| `payout_failed` | `on_payout` sees failure | token_id, amount, workshop, record index |
| `warranty_expired` / `expire_refund_failed` | First `expire` (amount may be 0) and any later refund retry / callback on failure | token_id, amount, issuer |

## 5.6 NEP notes

- **NEP-171:** use the standard `NonFungibleToken`. near-contract-standards 5.30.0 has **no transfer hook**, so the contract implements its own `NonFungibleTokenCore::nft_transfer`: it calls the inner `NonFungibleToken` transfer, then appends the holder record (ring of 50 + counter). Checked by the Cursor team in PR #1 (`docs/DECISIONS.md`).
- `nft_transfer_call` **disabled** in the MVP (panics). It would make the receiver the temporary owner until `nft_resolve_transfer`, which opens a claim-during-transfer window. Explained in the README.
- **NEP-178 approvals:** kept for standards compatibility but restricted: an approved account may transfer, never `pay_service`; `pay_service` is rejected while any approval exists ("Token is listed"); approvals are cleared on transfer (standard behaviour, tested in T24).
- **NEP-177:** token metadata: `title` "E-bike EB-001", `description`, `extra` JSON (asset_type, serial_hash); no image required. Contract metadata (`nft_metadata`) is **constant, not stored**: name "Phylaxtag", symbol "PHYLAX", spec = the NEP-177 version string from near-contract-standards ("nft-1.0.0" per PR #1), icon null, base_uri null, reference null.
- **NEP-181:** `nft_tokens_for_owner` drives "My items" and the gated page.
- **No sub-accounts / access keys per token.** The track prompt suggests sub-accounts; we use a per-token ledger inside one contract instead, so there is no key for a seller to keep. This is a deliberate MVP choice, stated in the README.

# 6. Threat model

## 6.1 How control passes on transfer (README text)

This section is written so it can be pasted into the README as is.

1. There is exactly one source of truth for control: `nft_token(token_id).owner_id` in the NEP-171 contract.
2. Every money-moving owner action (`pay_service`) checks `predecessor == owner_id` **at call time**. Nothing is cached, and there is no session, allowance, sub-account or access key per token.
3. The warranty fund is a field of the token (`Asset.balance`), not of any account. A transfer changes `owner_id` and nothing else, so the fund, service history, cooldown clock and limits stay with the item.
4. Transfer clears all NEP-178 approvals, so a marketplace or helper the seller approved loses its right to move the token.
5. `nft_transfer_call` is disabled, so there is no window where a receiver contract is a temporary owner.
6. Money can only leave to an approved workshop (never the owner) or to the issuer after expiry. A failed payment returns to the **token**, so whoever owns the token then benefits, never the old owner.
7. The holder-only web page re-reads `owner_id` from the chain on every load and before every action, so the old owner sees "You are not the current holder" right after the sale.

**What stops the old owner keeping money or access:** they never had custody of the money (no withdrawal path), the only "key" is current ownership checked on every call, the per-token cooldown means they cannot burn a last claim and re-arm it by moving the token around, and claims are blocked while the token is listed.

## 6.2 Transfer threat model: how a seller could try to keep money or access

| ID | Seller attack | Mitigation | Test |
|---|---|---|---|
| S1 | Drain the fund with a colluding approved workshop right before the sale | Per-claim cap + per-token cooldown bound the drain; buyer sees balance, last claim and "next claim possible at" on the asset page before paying; holder log records `balance_at_transfer` | T01, T15, T25 |
| S2 | Front-run the sale: buyer pays off-chain, seller claims before transferring | Same bound as S1. Sale price is paid off-chain in the MVP (no on-chain escrow); README tells buyers to check the balance after transfer. Atomic escrow sale: out of scope | T15, T25 |
| S3 | Listing + claiming race: approve a marketplace, then claim before the buyer's purchase lands | `pay_service` rejected while any approval exists; to claim, the seller must revoke, which makes the listing fail | T23 |
| S4 | Keep an approval after the sale to pull the NFT back | Approvals cleared on every transfer (standard) | T24 |
| S5 | Claim through an approved (non-owner) account | `pay_service` is owner-only | T12 |
| S6 | `nft_transfer_call` tricks (seller-controlled receiver keeps temporary ownership, claims, then resolves the token back) | `nft_transfer_call` disabled in MVP | T13 |
| S7 | Keep calling `pay_service` after the sale | Owner checked at call time | T02 |
| S8 | Sell while a payout is in flight and hope a failed payout refunds to them | Callback restores the amount to the token balance, never to an account; record marked Failed | T19, T20 |
| S9 | Reset the cooldown by transferring to a second wallet | Cooldown stored per token, not reset on transfer | T16 |
| S10 | Owner is also an approved workshop and pays themselves | `workshop ≠ owner`; issuer picks workshops | T18 |
| S11 | Re-mint: create a second token for the same bike, or reuse a token_id; or **another** allowlisted issuer mints the same serial_hash (two warranty NFTs for one item) | token_id never reusable (no burn in MVP); serial_hash unique per issuer. Cross-issuer duplicate: residual risk; buyer checks the issuer shown prominently on the asset page matches the brand/seller | T28 (same issuer only); cross-issuer duplicate = accepted residual risk (see 6.3) |
| S12 | Expire timing: claim just before expiry; or a failed payout lands after `expire` | Claims allowed only before expiry, within cap/cooldown; a refund that lands after expiry is sent to the issuer by calling `expire` again | T07, T11, T21, T36 |

| ID | Seller attack | Mitigation | Test |
|---|---|---|---|
| S16 | Hide previous holders by many self-transfers (push them out of the 50-entry log) | `holder_total` counter shown in UI ("55 transfers, showing last 50"); `nft_transfer` events are permanent; service records unaffected | T37 |
| S17 | Last-moment claim before selling so the buyer must wait a full cooldown | Not preventable without a contract change; "next claim possible at" shown on the asset page and in the Sell flow; README disclosure (6.3) | T38 |
| S13 | Keep using the holder-only page after the sale (cached session) | Page re-checks on-chain owner on each load and action | T31 |
| S14 | Keep a key that controls the item's wallet | No per-token sub-account or key exists in this design | Design review (Hermes) |
| S15 | Add money later and claim it back (old `top_up` path) | `top_up` removed; mint excess refunded to issuer at mint only | T17 |

## 6.3 General threats and residual risk

| Threat | Mitigation | Residual risk |
|---|---|---|
| Owner withdraws to own wallet | No withdrawal function; owner cannot be payee | Low |
| Owner colludes with an allowlisted workshop (fake repairs) | Cap + cooldown + allowlist + expiry; issuer chooses workshops and parameters | **Real and accepted.** Bounded by the formula in 5.2. No co-sign by design (user decision). Stated in README |
| Fake issuer mints fake items | Issuer allowlist | Admin is a trust point |
| A second allowlisted issuer mints the same serial_hash (two warranty NFTs for one bike) | Issuer shown prominently on the asset page; buyer checks it matches the brand/seller | **Accepted for MVP**; uniqueness is per issuer only |
| No `remove_issuer` / `set_admin` in the MVP | None in the MVP; disclosed in README | **Accepted for the testnet MVP.** A leaked issuer key can mint indefinitely; a lost admin key cannot be recovered. Post-hackathon item |
| Reentrancy / promise ordering | State changed before the transfer promise; `#[private]` callbacks | Low once T10/T19/T22/T35 pass |
| Workshop or issuer account deleted (transfer fails) | Callback restores balance; record Failed; `expire` retryable | Low |
| Contract account keys / code upgrade by deployer | Testnet only; README states the deployer could upgrade | Trust assumption. Production would need locked keys [decision later] |
| Personal data on-chain | Only salted serial hash and note hash | Low; salt must stay off-chain |
| NFT taken as proof of authenticity or legal title | Clear statement in UI and README | Depends on users reading it |
| Storage exhaustion | Exact storage at mint; 50-record caps; holder ring; reserve | Low for demo |
| Seller claims just before the sale; buyer waits a full cooldown for their first claim (cooldown is per token and never resets) | UI shows "next claim possible at" on the asset page and in the Sell flow; README disclosure. No contract change (resetting the cooldown would reopen S9) | **Accepted.** Buyer can see it before paying |
| Holder log hides holders older than the latest 50 | Total counter + permanent events | Low; needs 50+ transfers (gas only) |

## Honest limits (must be in the README)

- Prototype on **NEAR testnet**, not a product.
- The NFT does **not** prove physical authenticity and is **not** a legal title.
- The link between NFT and bike (serial number / QR) is outside the contract.
- Nobody verifies that the repair really happened; an approved workshop can collude with an owner within the limits.
- No keeper or required off-chain component.

### Deliberately NOT in the MVP

Own token, marketplace or on-chain escrow sale, IoT oracle, price oracle (the fund is denominated in NEAR, not USD), dispute system, multiple asset types, cross-chain, NEAR Intents, `top_up`, issuer co-sign.

## 7. Test plan

Unit tests for logic; integration tests in the NEAR sandbox (e.g. `near-workspaces`) for real transfers and callbacks; web end-to-end tests for pages. All contract tests must pass before the demo is recorded. IDs T01-T14 are the v1 tests, updated; T15-T33 were added in v2; T34-T38 in v3; T39 in v3.1.

| ID | Test | What it proves | Pass criteria |
|---|---|---|---|
| T01 | balance_moves_with_nft | Fund and history are per token | Mint 10, pay 2, transfer to Budi: owner Budi, balance 8, 1 record |
| T02 | old_owner_cannot_pay | Control passes on transfer | Alice → Budi; Alice `pay_service` fails "Not owner"; Budi succeeds after cooldown |
| T03 | non_approved_workshop_rejected | Allowlist enforced | Fails "Workshop not approved" |
| T04 | cap_enforced | Per-claim cap | 4 NEAR with cap 3 fails "Over cap"; 3 succeeds |
| T05 | over_balance_rejected | Cumulative cap = remaining fund | Balance 1, pay 2 fails "Insufficient balance" |
| T06 | expiry_returns_funds_to_issuer | Remainder goes to issuer only | After expiry, random account calls `expire`; issuer +remainder; balance 0; Expired |
| T07 | pay_after_expiry_rejected | Expiry enforced | Fails "Expired" |
| T08 | only_allowlisted_issuer_can_mint | Issuer allowlist | Fails "Not an approved issuer" |
| T09 | second_expire_is_noop | No double refund; defined second call | Second `expire` returns 0, issuer balance unchanged, no event, status still Expired |
| T10 | failed_payout_restores_balance | Callback failure path | Payee account missing: balance restored on token **and `total_locked` restored by the same amount**, T26 invariant holds, record Failed, `payout_failed` emitted |
| T11 | expire_before_expiry_rejected | No early refund | Fails "Not yet expired" |
| T12 | approved_account_cannot_pay | Approvals cannot claim | Fails "Not owner" |
| T13 | transfer_call_disabled | No temporary-owner window | `nft_transfer_call` panics; owner unchanged |
| T14 | history_view_paginated | History view correct | Order and count correct for from/limit |
| T15 | cooldown_enforced | Automatic cooldown | Second claim inside cooldown fails "Cooldown active"; succeeds after |
| T16 | cooldown_not_reset_by_transfer | S9 blocked | Claim, transfer to own 2nd account, claim again: fails "Cooldown active" |
| T17 | no_top_up_path | `top_up` removed; no later deposits | Calling `top_up` fails (method not found); `total_locked` unchanged by any call except mint, pay/expire, and their failed-payout callbacks (T10, T19, T21, T22) |
| T18 | owner_cannot_pay_self | S10 blocked | Owner who is on the workshop list pays self: fails "Cannot pay yourself" |
| T19 | sale_mid_payout_failure_unit | Sale during payout, failure path (deterministic) | `pay_service` (balance 10→8) → `nft_transfer` to Budi → `on_payout` called with Failed: balance back to 10 on the token and `total_locked` +2 (invariant holds), Budi is owner, Alice received nothing, record Failed |
| T20 | sale_mid_payout_sandbox | Same, on a real sandbox chain | Send `pay_service` (to a missing account) and `nft_transfer` without waiting; whatever the receipt order: final owner Budi, balance 10, Alice +0, record Failed. Also success variant: workshop +2, balance 8 |
| T21 | failed_payout_after_expire | S12 / callback after expiry | Payout fails after `expire` ran: amount restored on the Expired token (balance and `total_locked`); next `expire` returns that amount and sends it to issuer; balance 0; status Expired throughout |
| T22 | failed_expire_refund_restores | Expire callback failure | Issuer account missing: balance **and `total_locked`** restored, T26 invariant holds, status stays Expired, `expire_refund_failed` emitted; a later `expire` retries the refund |
| T23 | listed_token_blocks_claim | S3 blocked | `nft_approve(market)` then `pay_service` fails "Token is listed"; after revoke it works |
| T24 | approvals_cleared_on_transfer | S4 blocked | After transfer, old approved account cannot transfer |
| T25 | holder_history_records_transfers | "Who held it" view | Holder log = issuer, Alice, Budi with times and balances 10, 10, 8 |
| T26 | storage_accounting_exact | Exact demo numbers | Fund exactly 10 NEAR; excess refunded; invariant holds; bytes and cost per mint/record printed **[TO BE MEASURED]** |
| T27 | history_cap_50 | Bounded storage | 51st `pay_service` fails "History full" |

| ID | Test | What it proves | Pass criteria |
|---|---|---|---|
| T28 | no_remint | S11 blocked | Same token_id fails "Token exists"; same serial_hash for same issuer fails "Serial already used" |
| T29 | mint_input_validation | Bad parameters rejected | Past expiry, cap 0, cap > fund, empty list, duplicate workshop, issuer as workshop, bad `token_id` length, bad `serial_hash`, small deposit: each fails (cooldown 0: see T34) |
| T30 | string_limits | Storage cannot be inflated | Over-long `work_type` / bad `note_hash` fail |
| T31 | gated_page_access (web e2e) | Only current holder opens the page | Before sale Alice sees owner page; after sale Alice sees "not the current holder", Budi sees it |
| T32 | sell_button_flow (web e2e) | Demo "Sell this item" | Click sell → Sell dialog shows "next claim possible at"; owner Budi, fund unchanged, holder log +1, Alice locked out |
| T33 | readme_newcomer_run (manual) | README reproducible | Asa-Hermes follows README only; every "do X - see Y" step matches; failures filed as issues |
| T34 | mint_rejects_zero_cooldown | B1 validation | `mint` with `cooldown_ns = 0` fails "Cooldown must be > 0" |
| T35 | same_block_double_claim_rejected | No double spend without a pending flag | Two `pay_service` sent without waiting (same block where possible): exactly one succeeds, the other fails "Cooldown active"; balance deducted once |
| T36 | expire_empty_fund_marks_expired | B2: empty token becomes Expired | Fund used to 0; after expiry `expire` returns 0, no transfer, status Expired, `warranty_expired` with amount 0 |
| T37 | holder_log_full_never_blocks_transfer | B3 ring buffer | 55 transfers all succeed; log has 50 entries (latest, oldest first); `holder_total` = 56 incl. mint; storage stays bounded **[TO BE MEASURED]** |
| T38 | ui_status_and_next_claim (web e2e) | B2/B4 UI | Labels Active / Empty (fund used up) / Expired, close pending / Expired shown correctly; "next claim possible at" on asset page and Sell dialog |
| T39 | mint_rejects_contract_as_workshop | Payouts cannot loop back into the contract (T26 invariant) | `mint` with the contract's own account in `workshops` fails "Bad workshop list" |

Mapping v2 → v3: T09 renamed (reject → no-op), T21/T22 updated, T29 points to T34; T34-T38 are new; T39 added in v3.1. **Total: 39 tests** (35 contract tests: T01-T30, T34-T37, T39; 3 web e2e: T31, T32, T38; 1 manual: T33).

## 8. Web pages

Static web app (e.g. Vite/React or Next.js static export) with NEAR Wallet Selector on testnet. Plain language for newcomers; technical details on a "How it works" page.

| Page | Content | Notes |
|---|---|---|
| Asset page `/asset/EB-001` (public) | Name, **issuer (shown prominently)**, status (**Active / Empty (fund used up) / Expired, close pending / Expired**), remaining fund, expiry, cap, cooldown and "next claim possible at", approved workshops, last 5 services; "Close warranty" button for anyone when expired with balance > 0 | Readable without login; this is the buyer's pre-purchase check |
| History `/asset/EB-001/history` (public) | **Funded** (mint deposit), **paid out** (each service: workshop, amount, type, status; expiry refund), **who held it** (latest 50 holders with dates and balance at transfer, plus "N transfers in total"), links to explorer | Reads `get_service_history` and `get_holder_history` |
| Holder-only page `/asset/EB-001/owner` (gated) | "Pay workshop from warranty fund" form, "Sell this item" button (dialog shows "next claim possible at" for the buyer), service note uploads (hash computed in browser) | Opens only if the signed-in account = on-chain owner, re-checked on each load and action. Others see "Only the current holder can open this page" |
| "Sell this item" (demo) | Enter buyer account → `nft_transfer`; then shows before/after: owner, fund, history count | Shows the fund moving to the new owner live |
| Issuer page | Mint form: ID, serial (salted and hashed in browser), fund, expiry, cap, cooldown, workshops; "Demo mode" preset fills short cooldown/expiry | Raw serial and salt never sent on-chain |
| Workshop page | Payments received | No required action |
| How it works | Section 6.1 text, function table, explorer links, honest limits | For judges and AI pre-screeners |

**Gate caveat:** in a static site the gate controls the interface; the real lock is the on-chain owner check. If any content behind the gate must stay secret (e.g. service receipts), it needs a small verifier that checks a signed message against the on-chain owner. Not in the MVP unless decided (section 14).

## 9. Team plan (three teams)

| Team | Lead / members | Owns | Does not |
|---|---|---|---|
| **Cursor** | Cursor coding agents | **Only code writer.** Contract, tests, web, deploy scripts, in **one repo with LICENSE** (new public repo recommended, not mandatory per Q-003); every change via pull request | Edit the spec; hold keys; deploy with keys it does not have; submit |
| **Grok** | Lead **Pharita**; **Haerin** = testnet accounts and deploy, together with the user; **Asa-Grok** = demo video + X post draft | Spec document (this file), schedule, hackathon rules research, submission checklist | Write code; post or submit without the user |
| **Hermes** | Lead **Jiwoo** = red-team review of spec and code; **Sohee** = README, pitch, submission text, video script; **Asa-Hermes** = newcomer tester who follows the README step by step | Review findings (as issues/PR comments), README text (delivered to Cursor as PR content), T33 | Edit the spec directly; merge code |

### Rules

- **The spec doc is changed only by Pharita.** Others send change requests.
- **Code is changed only by the Cursor team, via pull requests.** Hermes reviews; the user (or a reviewer the user names) merges.
- **Secrets and keys are held only by the user.** No agent receives a seed phrase or private key. Haerin works with the user, who signs transactions in their own wallet/CLI.
- **The user decides and presses the final submit**, after reading the exact payload. The user also publishes the X post and makes the repo public.
- Unknowns go into section 14 as [UNVERIFIED] until someone checks them; nobody invents versions, numbers or facts.

## 10. Day-by-day schedule

Target submit **around 12 Nov 2026**; start **no later than about 8 Nov**. Dates below are the latest-start plan; starting earlier just adds buffer. Deadline: 14 Nov 23:59 UTC = 15 Nov 06:59 WIB.

| Day | Latest date | Focus | Owner(s) | Done when |
|---|---|---|---|---|
| 0 | ≤ 8 Nov | User decisions recorded; spec v3.3 approved; repo (new public repo recommended) + LICENSE; testnet accounts (admin, issuer, Alice, Budi, 2 workshops) created by the user; pin near-sdk version and fallback | User, Pharita, Haerin, Cursor | Repo exists with LICENSE and pinned `Cargo.toml`; accounts listed (names only) in README draft |
| 1 | 9 Nov | Core contract: `new`, `mint` (exact storage), `pay_service` with all limits, `nft_transfer` wrapper, views; basic tests | Cursor | Builds; T01-T05, T08, T12-T18, T28-T30, T34-T35, T39 pass |
| 2 | 10 Nov | `expire` + callbacks; sale-mid-payout tests; storage accounting test; Jiwoo red-team review | Cursor, Jiwoo | T06-T11, T19-T27, T36-T37 pass; storage figures measured; review issues triaged |
| 3 | 11 Nov | Web pages (asset, history, gated, sell, issuer); testnet deploy; README; video recording; X post draft | Cursor, Haerin + user, Sohee, Asa-Grok | T31-T32, T38 pass; demo runs on testnet; README "do X - see Y" complete |

| Day | Latest date | Focus | Owner(s) | Done when |
|---|---|---|---|---|
| 4 | 12 Nov | Buffer only: fixes, newcomer test, checklist, submit | Asa-Hermes, Pharita, user | T33 done; section 12 checklist ticked; user submits |

## 11. Demo video script (60-90 s)

| Time | Screen | Voice-over (draft) |
|---|---|---|
| 0-6 s | Bike photo + title | "This e-bike has its own warranty fund. Sell the bike, and the fund and service book move with it." |
| 6-14 | Problem text | "Warranties usually die at resale. Service records get lost." |
| 14-26 | Issuer page: mint EB-001, 10 NEAR, cap 3, demo cooldown | "The seller mints an NFT and deposits 10 NEAR once. It picks the approved workshops and the limits." |
| 26-38 | Holder page: pay 2 NEAR to workshop A; history updates | "The owner pays a workshop straight from the fund. One call. Balance: 8." |
| 38-46 | Try 4 NEAR / another workshop / again too soon → rejected | "Over the cap, a non-approved workshop, or too soon? The contract says no. The owner can never withdraw." |
| 46-60 | "Sell this item" → Budi; history shows new holder; Alice opens holder page → locked | "Alice sells to Budi. The 8 NEAR and the history move with the bike. Alice is locked out instantly." |
| 60-72 | Demo token past expiry → `expire` → issuer receives remainder | "When the warranty ends, anyone can close it. The rest goes back to the seller." |
| 72-85 | Test run (sale mid-payout passing), explorer, repo | "Tested even for a sale in the middle of a payout. Built on NEAR testnet." |

Asa-Grok records; Sohee finalizes the script. The X post must say it was built at NEAR HACKATHON and tag @NEARProtocol and #NEARHackathon (optionally @NearLegion, @NearBuilders); the user publishes it.

## 12. Hackathon rules and submission checklist

Source: live HACKATHON.md fetched 2026-11-04 (local copy `/workspace/hk_now.md`). The live file may change; Pharita re-reads it before submit.

| Requirement | Detail | ✓ |
|---|---|---|
| Code inside the window | Q-001: code written since the 19 Oct 2026 announcement counts; window ends 14 Nov 2026. Our build days (8-12 Nov) are inside the window. **New repo: OPTIONAL** (Q-003: "No. Dedicated branch, or tags pre-hackathon and hackathon-submission, plus a README section linking the diff."). Recommended for us: a new repo created in the window | ☐ |
| Public repo | Public before **14 Nov 23:59 UTC (15 Nov 06:59 WIB)**; stays public through judging; one submission per repo | ☐ |
| LICENSE | Any license (MIT fine); file in repo | ☐ |
| README "do X - see Y" | Claude Code and Codex pre-screen READMEs (Q-017): what it is, problem, how to try it as steps, no marketing | ☐ |
| Disclosure | Libraries used and AI-generated code disclosed in README (Q-005) | ☐ |
| X post with video | Built at NEAR HACKATHON, @NEARProtocol, #NEARHackathon, video attached; published by the user | ☐ |
| Reproducible demo | Local demo is OK only with public repo + reproducible README + X video; hosted demo is a plus (keep up to 18 Nov, ideally 25 Nov) | ☐ |
| Tracks | `nfts`, optionally `realworld` / `open` (ids verified against live HACKATHON.md on 2026-11-04) | ☐ |
| Payout account | Mainnet NEAR account created by the user; never invented | ☐ |
| Two-project rule | Max 2 different projects per person; each README names the other one if any | ☐ |
| One submission, no edits | Submit only when repo is public and post is live; the user confirms the exact JSON payload; same requestId on retry | ☐ |
| Human-decided fields | Project name, lead name, email, team size, tracks, payout account, repo link, post link, description (20-2000 chars), consent | ☐ |

## 12.1 Submission API contract (verbatim from live HACKATHON.md, 2026-11-04)

`POST https://nearhackathon.com/api/applications` with `Content-Type: application/json`. Body up to 12000 bytes. Send exactly these fields:

| Field | Type | Rule | For Phylaxtag (ours, not verbatim) |
|---|---|---|---|
| requestId | string | UUID, generated once per submission and reused on retries | Generated once by Pharita at submit time; reused on retries |
| projectName | string | 2-80 characters | "Phylaxtag" (decided name; user confirms in the payload) |
| leadName | string | 2-80 characters | User decides |
| email | string | valid email, max 160 characters, at most 2 submissions per email | User decides; not committed anywhere |
| size | string | one of `1`, `2 - 4`, `4 - 6`, `6 - 8`, `8+`, exactly as written | "1" unless the user says otherwise |
| tracks | array of strings | one or more of `defi`, `thesis`, `privacy`, `business`, `swarms`, `security`, `agents`, `realworld`, `games`, `nfts`, `memecoins`, `social`, `open`, each listed once | ["nfts"], optionally "realworld" / "open" |
| nearAddress | string | 2-64 characters, lowercase, matching `^(([a-z\d]+[-_])*[a-z\d]+\.)*([a-z\d]+[-_])*[a-z\d]+$` | Mainnet account created by the user in their own wallet; agents never handle keys |
| projectUrl | string | http(s) URL of the public repository, max 300 characters, unique: each repository is submitted once | The public repo `phylaxtag` (URL set when created) |
| demoUrl | string | http(s) URL of the public demo post, max 300 characters | The X post with the video |
| idea | string | 20-2000 characters | Section 13 text |
| consent | boolean | `true`, only after the human agreed to the contact notice | true only after the user agrees to the notice below |

Do not add other fields. In particular `website` is a spam trap: any non-empty value rejects the submission.

| Status | Meaning | What to do |
|---|---|---|
| 201 | Submission saved | Done. Tell the human. |
| 400 | A field failed validation | Fix the fields listed in `fields` of the response and resend with the same requestId. |
| 403 | Submissions are not open, or the request looked cross-site | Tell the human. Do not retry. |
| 409 | Repository already submitted, or this email already has 2 submissions | The project was already submitted, or the human reached the limit. Ask the human; do not change the link or the email to get around it. |
| 413 | Body too large | Shorten the description. |
| 503 | Storage temporarily unavailable | Wait a minute and resend with the same requestId. |

Consent notice, to be read to the user verbatim:

> "I agree that the organizers may use these details to review our submission and contact us about this hackathon."

Other verbatim rules:

- Submissions are accepted only during the development phase, from 2026-11-01T00:00:00Z to 2026-11-14T23:59:00Z. Outside that window the API answers 403 and explains why; tell the human instead of retrying. A submission cannot be edited, so send it when the repository is public and the demo post is up.
- Generate `requestId` once, with `uuidgen | tr A-Z a-z` or `node -e 'console.log(crypto.randomUUID())'`. Keep it: a retry after a network error or a 503 must reuse the same value, and the server then treats it as the same submission. Do not commit the payload; it contains personal data.
- Send the submission. Print the full JSON payload first and wait for a yes. A second submission of the same repository is rejected, so there is one attempt per project.
- The repository may stay private while the team builds, but it must be public by 2026-11-14T23:59:00Z. Judges cannot open a private one. Claude Code and Codex screen every project before the human jury

decides (Q-017), so write the README for a machine reader as well as a human one: what the product is, the problem it solves, and how to try it as "do X - see Y" steps a judge can follow. No marketing.

- A 201 response means the submission is saved: tell the human. If they prefer the form, the same fields are at https://nearhackathon.com/#apply.

**Notes (ours).** Track ids verified against live HACKATHON.md on 2026-11-04; if the API returns 400 listing fields, fix and resend with same requestId. The reviewer report that validation "only accepts `open`" was **not** confirmed: the live file lists `nfts` as a valid id. Never commit the payload (personal data). Submit once, only after the user confirmed the exact payload; the submission cannot be edited. Repo must be public by 2026-11-14T23:59Z.

### 12.2 README outline

1. What it is (one paragraph + tagline), including: "Phylax (φύλαξ) is Greek for 'guardian': the warranty is a guardian that lives on the item and transfers with it."
2. Try it: "Open <link> - see EB-001 with 8 NEAR left and 1 service record"
3. Run tests: "Run `cargo test` in `contract/` - see 35 tests pass"; "Run the web e2e tests in `web/` - see 3 tests pass" (39 total with the manual newcomer run T33; exact web command set by Cursor)
4. Deploy: "Run `./scripts/deploy.sh <your-testnet-account>` - see the contract id"
5. How control passes on transfer (section 6.1) and transfer threat model (6.2)
6. Functions and limits; storage figures (measured)
7. Honest limits
8. Libraries used, AI-generated code disclosure, other project (if any)
9. License

## 13. Submission text (English)

Draft by Sohee, owned by the user. Recounted programmatically for v3 (Python `len()` on the exact string below): **972 characters** including the two blank-line paragraph breaks (4 newline characters), 972 UTF-8 bytes; **970** if paragraphs are separated by single newlines (this explains the reviewer count of 966). Target ≤ 1100; the API allows 20-2000. Count again on the exact `idea` string before submitting.

> Phylaxtag: the product owns its warranty.
>
> Phylaxtag turns a physical item (MVP: one e-bike) into a NEP-171 NFT on NEAR testnet that carries its own warranty fund and service history. The issuer mints the NFT with a one-time deposit and sets approved workshops, an expiry, a per-claim cap and a cooldown. The owner pays an approved workshop straight from the fund with one call; the contract enforces every limit, and the owner can never withdraw the fund. The fund is stored per token, not per owner, so when the NFT is sold the remaining fund and full history move to the buyer and the seller loses access at once. After expiry, anyone can return the remainder to the issuer. A holder-only page and a per-NFT history (funded, paid out, who held it) make this visible. Tests cover a sale in the middle of a payout, failed payouts and each seller trick in our threat model.
>
> Limits: testnet prototype. The NFT does not prove physical authenticity and is not a legal title.

## 14. Open items and unverified claims

### 14.1 Decided

| # | Item | Decision |
|---|---|---|
| D1 | `top_up` | Removed (user, 4 Nov 2026) |
| D2 | Claim limits | Automatic, contract-only; no co-sign (user, 4 Nov 2026) |
| D3 | Zero cooldown / in-flight guard | `cooldown_ns > 0` required; no pending flag (5.2) |

| # | Item | Decision |
|---|---|---|
| D4 | Expire on empty fund; second expire | Always marks Expired; second call is a no-op returning 0 (5.1) |
| D5 | Holder log full | Drop-oldest ring of 50 + total counter; transfer never fails (5.3) |
| D6 | Per-token balance vs sub-accounts | Per-token balance in one contract (justified in 5.6); disclose in README |

## 14.2 Still open

| # | Item | Status / owner |
|---|---|---|
| O1 | Standing OK for Cursor to push PR branches (user rule: no push without the user's command) | **Open, user.** Making the repo public and merging stay with the user |
| O2 | Holder-only page: on-chain owner check in UI vs signed-message checker for secret content | **Open, user.** Proposed: UI gate + on-chain checks |
| O3 | Holder list on-chain (capped ring) vs indexer | **Open, user.** Pharita recommends on-chain (as specified) |
| O4 | Upgrade risk: the deployer can change contract code | **Open, user.** Proposed: disclose in README; no key locking on testnet |
| O5 | Display name | **Decided (user, 4 Nov 2026): Phylaxtag.** Earlier candidates: CareTag (working name), Covered, WarrantyTag, ServiceBook; CareTag was dropped because of existing trademark use of "Caretag" in medical RFID. X @phylaxtag and phylaxtag.com still to be registered **[UNVERIFIED]** |
| O6 | Service history full (50) blocks further claims; remainder returns at expiry | Open, Pharita. Kept for MVP (audit trail); alternative: higher cap |
| O7 | Minimum cooldown for real tokens (beyond > 0) | Open, Pharita with issuer; demo uses a short value |
| O8 | Team size and lead name for the form | User decides; default size "1" |
| O9 | `remove_issuer` / `set_admin` (key leak / admin recovery) | Post-hackathon item; accepted for MVP (6.3) |

## 14.3 Claims and facts

| # | Claim | Status |
|---|---|---|
| F1 | Similar projects | Reported by reviewer Jiwoo; not independently verified by Pharita: only EVM/Tezos prior art (aynp/nft-warranty, 2022, Goerli; tezos-warranty) and a 2024 SSRN paper about NFT warranties on NEAR; no working NEAR implementation found. Keep the wording "to our knowledge, a working NFT-warranty implementation on NEAR"; never "first". **[UNVERIFIED]** |
| F2 | Code window | Q-001 in live HACKATHON.md: code since the 19 Oct 2026 announcement counts; work on 8-12 Nov is in the window (verified in the file) |
| F3 | Historic judging criteria (technical implementation, design, impact, originality, sustainability) | Reviewer-reported; not in HACKATHON.md **[UNVERIFIED]**. Used only as pitch ordering |
| F4 | "88 submissions" figure | **Do not use** anywhere **[UNVERIFIED]** |
| F5 | near-sdk / near-contract-standards versions and NEP-171 override hooks | **Resolved (PR #1):** near-sdk 5.30.0, near-contract-standards 5.30.0, near-workspaces 0.24.0; no transfer hook, own `nft_transfer` override (5.6) |
| F6 | Pyth stopped supporting NEAR (per Jiwoo) | **[UNVERIFIED]**. Not needed: the MVP uses no price oracle |
| F7 | Storage cost per mint / record / holder slot | **[TO BE MEASURED]** by T26, T37 |
| F8 | Live validation "only accepts `open`" (reviewer) | Not confirmed: live HACKATHON.md lists `nfts`; handle a 400 per 12.1 |

Document safety: this file contains no keys, seed phrases or secrets. Do not add any to the repo or to agent prompts.